System Design Interview

Do's, Don'ts & a Practical Interview Playbook

For senior engineers, the goal is not to list technologies. It is to turn an ambiguous problem into a clear design, justify trade-offs, and reason about scale and failure.

1. The 10 Do's

1. Clarify requirements — Ask about core features, users, scale, latency, consistency and availability. Separate must-haves from optional features.

2. Estimate scale early — Roughly estimate requests per second, data volume, storage growth and read/write ratio. Use estimates to justify decisions.

3. Draw the high-level architecture first — Show clients, APIs, services, databases, caches, queues and external systems before diving into details.

4. Explain the data model and access patterns — Identify entities, keys, indexes, query patterns and data ownership. Choose databases based on workload, not popularity.

5. Deep-dive into the critical component — Focus on what makes the problem hard: partitioning, routing, ranking, concurrency, consistency or event processing.

6. Discuss failure scenarios — Consider node failures, duplicate messages, stale caches, timeouts, retries and downstream outages.

7. State trade-offs explicitly — Explain why you chose an option, what it costs, and when an alternative would be better.

8. Design incrementally — Start simple; add caching, replication, sharding or asynchronous processing only when requirements justify them.

9. Address operations and observability — Mention metrics, logs, tracing, alerts, capacity planning, deployment and recovery.

10. Lead the conversation — Explain your reasoning, check alignment with the interviewer and manage time. Treat the interview as collaborative design.

2. The Don'ts

Don't	Why it hurts	Better approach
Start designing immediately	You may solve the wrong problem.	Clarify requirements and assumptions first.
Jump straight to technologies	Choices lack justification.	Explain which requirement each technology satisfies.
Overengineer for enormous scale	Adds unnecessary complexity.	Scale the design in response to stated needs.
Ignore consistency	Can break business correctness.	Define consistency expectations per operation.
Discuss only the happy path	Real systems fail.	Cover retries, timeouts, duplicates and recovery.
Use buzzwords without mechanisms	Doesn't demonstrate depth.	Explain how sharding, replication or failover works.
Assume queues guarantee exactly-once effects	End-to-end effects can still duplicate.	Use idempotency, deduplication and clear delivery semantics.
Spend too long on one component	You may not finish.	Time-box deep dives.
Defend decisions rigidly	Blocks collaboration.	Explain trade-offs and adapt to new requirements.

3. A Practical 45-Minute Framework

Time	Focus	What to cover
0–5 min	Requirements & scope	Functional requirements, constraints and assumptions.
5–10 min	Scale & data model	Traffic estimates, storage needs, entities and access patterns.
10–18 min	High-level architecture	Components, responsibilities and request flows.
18–33 min	Deep dive	The hardest part: partitioning, concurrency, consistency or algorithms.
33–40 min	Reliability & trade-offs	Failures, retries, bottlenecks, security and operations.
40–45 min	Summary & evolution	Recap key choices, limitations and possible improvements.

4. Questions That Distinguish a Senior Engineer

Why this? Why choose this database, queue, cache or protocol?

What breaks first? Identify likely bottlenecks and how you would detect them.

What happens on failure? Consider partial failures, retries, stale state and recovery.

What are the trade-offs? What becomes harder, slower or more expensive with this choice?

When would you change the design? Define the scale or requirement that justifies added complexity.

5. Example: Explain a Decision with Mechanisms

Weak: “I'll use Redis for nearby friends.”

Stronger: “I'll maintain online presence with a TTL and store locations in a Redis GEO index for radius queries. A heartbeat timeout determines when presence is stale. Since Pub/Sub events can be missed, reconnecting clients refresh their presence snapshot. If durable event replay is required, I'll consider Redis Streams or another durable messaging platform.”

This answer demonstrates the architecture, the mechanism, failure handling and the trade-off.

6. Pre-Interview Checklist

[ ] I clarified functional and non-functional requirements.

[ ] I stated assumptions and estimated scale.

[ ] I drew the high-level architecture and request flow.

[ ] I explained the data model and access patterns.

[ ] I justified major technology choices.

[ ] I covered concurrency, consistency and failure scenarios.

[ ] I identified bottlenecks and scaling strategies.

[ ] I discussed security, observability and operations.

[ ] I explained trade-offs rather than just naming technologies.

[ ] I summarized the design and its limitations.

Remember: A strong system design interview is a structured conversation. Clarify the problem, make defensible choices, explain what happens under load and failure, and adapt when requirements change.